

Data Stream Pipeline

Daniele Riboni

Università degli Studi di Cagliari

Dept. of Mathematics and Computer Science

Simulation: Smart Wine-Bottling Line

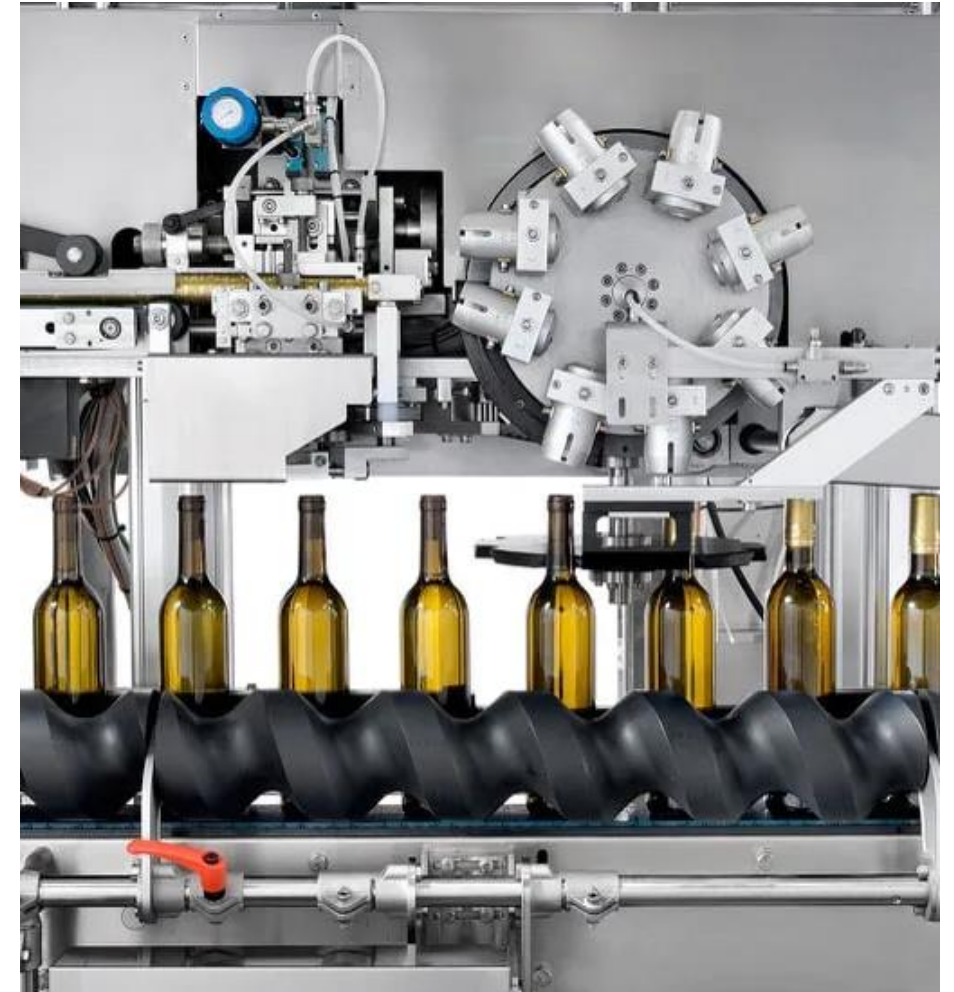
Objective: monitor quality and diagnostic data, detect anomalies, store statistics

Machine 1 — Filling Machine

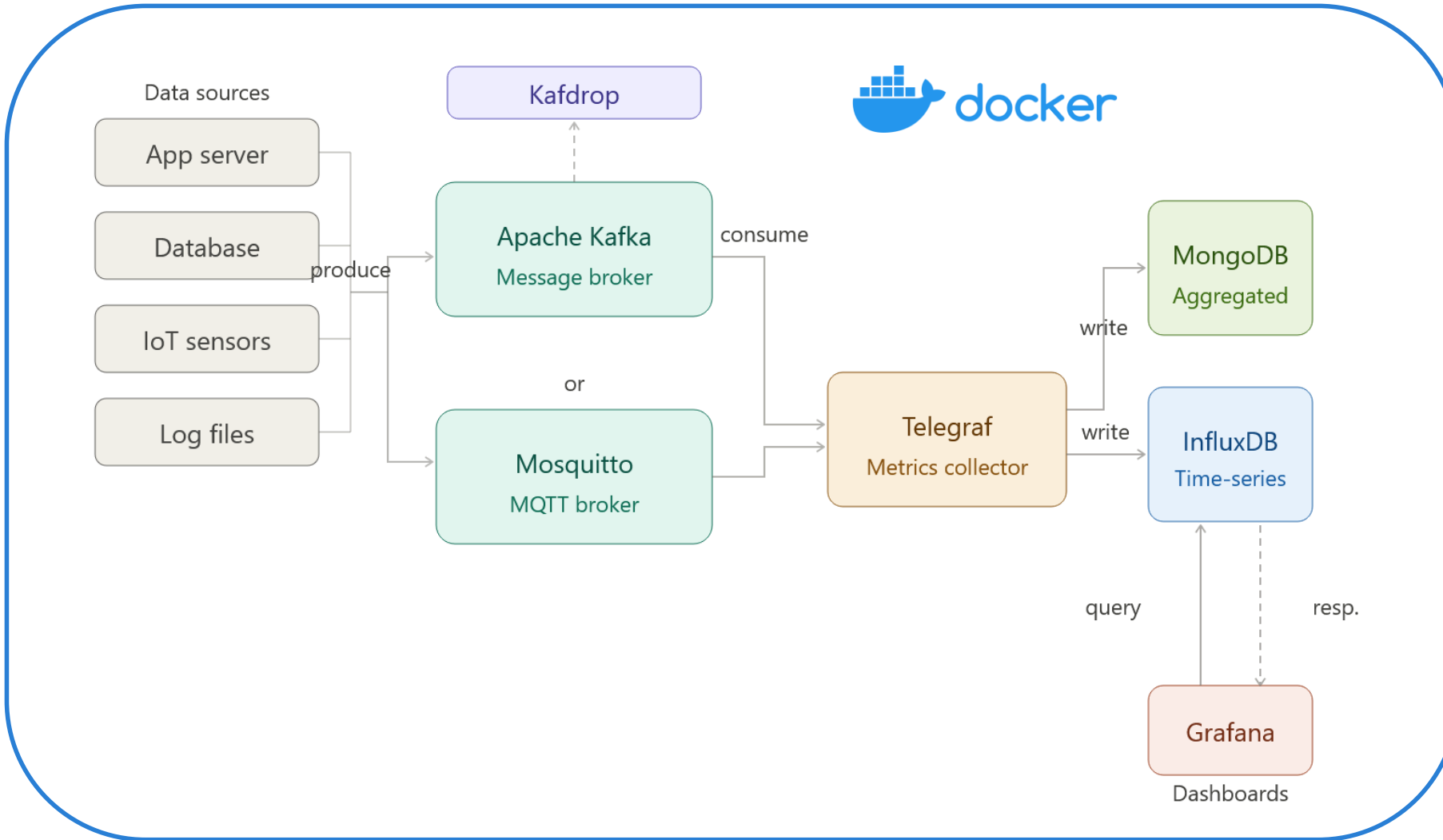
- Liquid temperature sensor ($^{\circ}\text{C}$)
- Circuit pressure sensor (bar)
- Counter of bottles filled per minute (units/min)

Machine 2 — Capping Machine

- Capping force sensor (N)
- Motor vibration sensor (mm/s)
- Bottles capped per minute (units/min)



A pipeline to handle data streams



Deploy the pipeline

- Launch Docker Desktop
- Unzip `newpipeline.zip` in a new folder
- From that folder:
`docker compose up -d --build`
- Now the pipeline should be up and running

producer.py publishes sensor data

Filling machine publishes to topic
`'sensors.riempitrice'`

Values produced:

- liquid temperature: °C
- circuit pressure: bar
- bottles filled per minute: units/min

Temperat. normal range: 18-22 °C

```
{  
  "machine_id": "riempitrice",  
  "timestamp": "2026-05-21T16:00",  
  "temperatura_liquido_C": 33.29,  
  "pressione_circuito_bar": 1.619,  
  "bottiglie_riempite_pz_min": 49  
}
```

Abnormal behavior: cyclic overheating

- Every 60 seconds, temperature anomaly starts and it lasts 20 seconds
- During the anomaly, temperature can rise up to about 45 °C

producer.py publishes sensor data

Capping Machine publishes to topic
`'sensors.tappatrice'`

Values produced:

- capping force: N
- motor vibration: mm/s
- bottles capped per minute: units/min

Vibrat. normal range: 0.5–2.0 mm/s

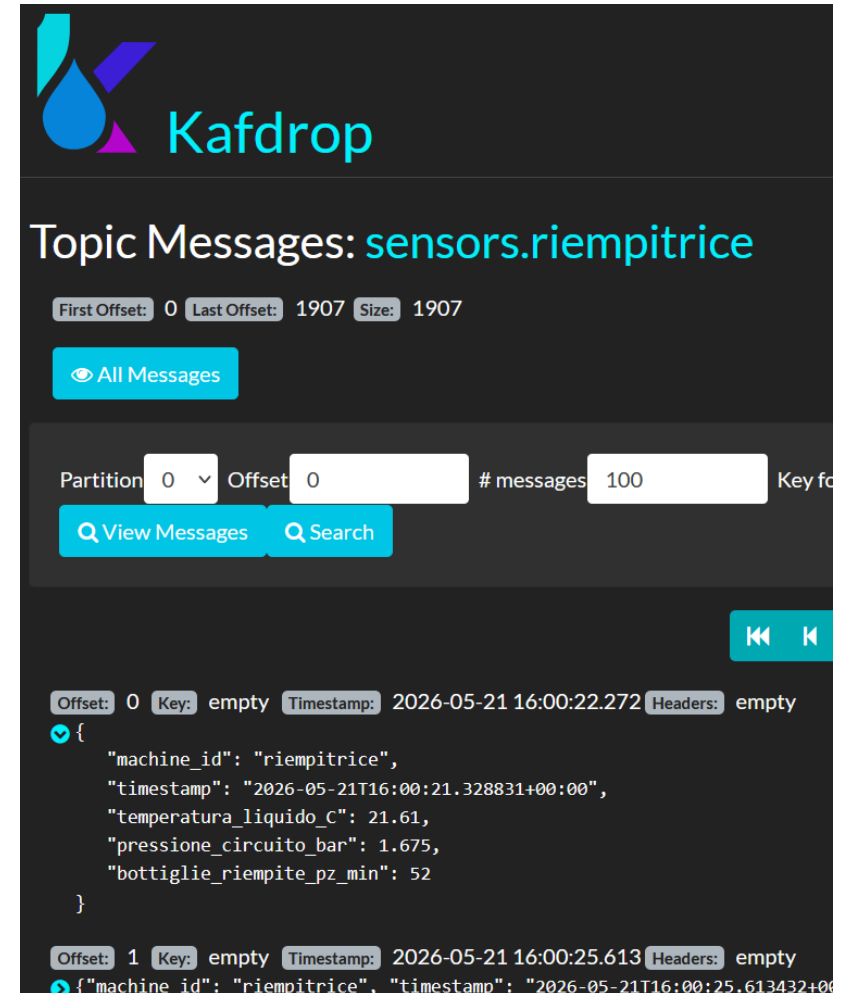
```
{  
  "machine_id": "tappatrice",  
  "timestamp": "2026-05-21T16:00",  
  "forza_tappatura_N": 107.51,  
  "vibrazione_motore_mm_s": 1.691,  
  "bottiglie_tappate_pz_min": 40  
}
```

Abnormal behavior: cyclic excessive vibration

- Every 90 seconds, vibration anomaly starts and it lasts 25 seconds
- During the anomaly, vibration can rise up to about 15 mm

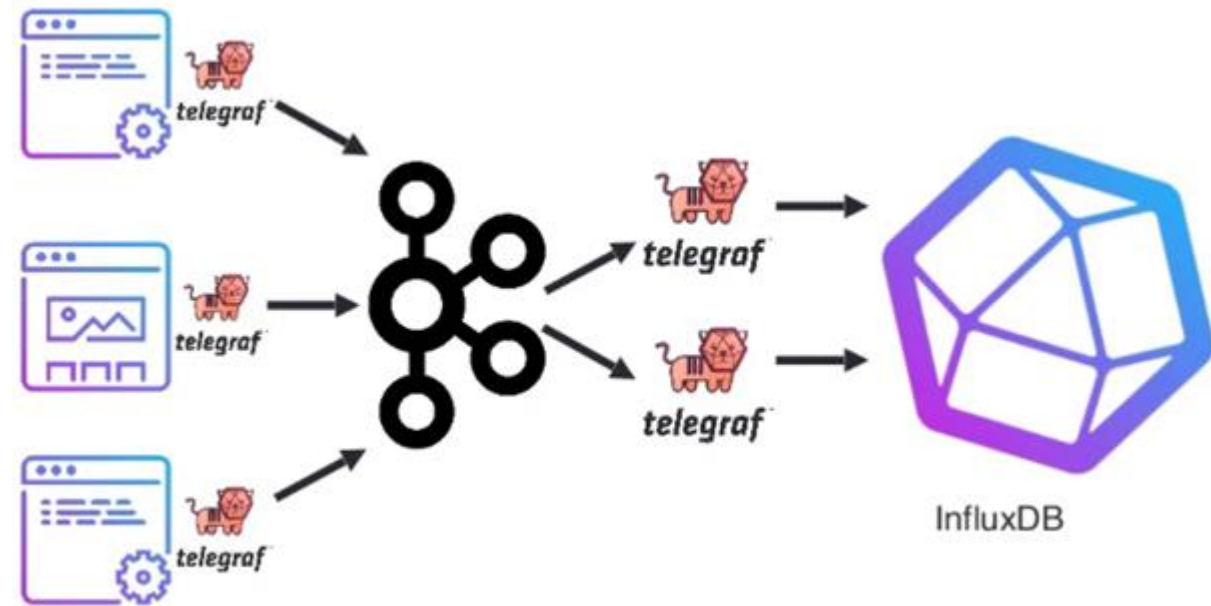
Look at Kafka with Kafdrop

- **Web UI for Apache Kafka:** Kafdrop provides a browser-based interface to explore and monitor Kafka clusters
- **View Topics & Messages:** Allows users to browse Kafka topics, partitions, and inspect real-time messages
- **Monitor Consumer Groups:** Displays consumer group offsets and lag, helping ensure data is being consumed properly
- **Inspect Cluster Metadata:** Visualizes brokers, topics, partitions, and configurations in an easy-to-read format
- **Troubleshooting & Debugging Tool:** Essential for developers and operators to debug Kafka flows without writing consumers



Telegraf

- **Data Collection Agent:** Telegraf reads data from brokers (Kafka, MQTT), parses JSON messages, transforms data into metrics, writes data (to InfluxDB, MongoDB)
- **Kafka Integration:** Acts as a producer or consumer to send or receive streaming data via Kafka topics
- **Basic Real-Time Processing:** Applies filtering, parsing, and simple transformation and aggregation
- **InfluxDB Output:** Forwards cleaned and structured data to InfluxDB for storage, analysis, and visualization



(InfluxDB does not natively consume Kafka topics)

Telegraf and Kafka

Kafka

- High-throughput, fault-tolerance, durability
- Supports pub/sub and queueing models

Telegraf

- Lightweight
- No storage, minimal buffering (no long-term retention, it is a pure forwarder)
- 200+ input plugins, 50+ output plugins (InfluxDB, Kafka, MongoDB, ...)
- Built-in aggregation/processing (e.g., downsample metrics)

telegraf.conf

```
[agent]
  interval = "5s"
  round_interval = true
  metric_batch_size = 1000
  metric_buffer_limit = 10000
  collection_jitter = "0s"
  flush_interval = "5s"
  flush_jitter = "0s"
  precision = ""
  hostname = "telegraf"
  omit_hostname = false
```

- Collects metrics every 5 seconds
- Sends collected metrics every 5 seconds
- Sends metrics in batches, up to 1000 at a time
- Temporarily stores up to 10000 metrics if InfluxDB is unavailable
- Uses hostname 'telegraf' to identify the source
- Adds the hostname field to the collected metrics

telegraf.conf

```
[[inputs.kafka_consumer]]
  brokers = ["winery-kafka:29092"]
  topics = ["sensors.riempitrice"]
  consumer_group = "telegraf-riempitrice"
  offset = "newest"
  name_override = "riempitrice"
  data_format = "json_v2"

[[inputs.kafka_consumer.json_v2]]
  [[inputs.kafka_consumer.json_v2.tag]]
    path = "machine_id"

[[inputs.kafka_consumer.json_v2.field]]
  path = "temperatura_liquido_C"
  type = "float"
```

- Connects Telegraf to the Kafka broker 'winery-kafka:29092'
- Subscribes to the topic 'sensors.riempitrice'
- Reads only new messages
- Parses incoming messages as JSON
- Saves data in InfluxDB as measurement 'riempitrice'
- Uses machine_id as a tag to identify the source machine

telegraf.conf

```
[[outputs.influxdb_v2]]
  urls = ["http://winery-influxdb:8086"]
  token = "winery-super-secret-token"
  organization = "winery"
  bucket = "sensors"
  [outputs.influxdb_v2.tagdrop]
    aggregated = ["true"]
```

- Sends Telegraf metrics to InfluxDB
- Connects to:
http://winery-influxdb:8086
- Uses an authentication token
- Writes data to:
 - Organization: winery
 - Bucket: sensors
- Excludes metrics marked as:
aggregated = true

Influxdb

- **Time-Series Database:** Purpose-built to store and query high-volume time-stamped data like metrics, events, and logs
- **High-Performance Storage:** Optimized for fast writes and queries, making it ideal for real-time monitoring and alerting
- **Powerful Query Language** (Flux/InfluxQL/SQL): Enables advanced analytics, data transformation, and alert rule creation
- **Feeds Data to Grafana Dashboards:** Acts as the backend data source for Grafana, providing rich visualizations and insights

Influxdb logical model

Organization

- └ Bucket (with retention policy)
 - └ Measurement
 - └ Tags (metadata, indexed)
 - └ Fields (data)
 - └ Timestamp

Bucket: is the top-level container for time series data. Each bucket:

- Has a **retention policy** (how long data is kept)
- Belongs to an **organization**; roughly equivalent to **a database**

Measurement: is similar to **a table**

- Each measurement represents a stream of data points with the same schema

Influxdb logical model

Tags are key-value pairs used to store *metadata*

- Always stored as strings
- Indexed (efficient for filtering queries)
 - Example: room = "kitchen"

Fields are key-value pairs that contain the actual data values

- Fields are not indexed, and are typically float, int, string
 - Example: temperature = 23.4

Time is a timestamp (usually in nanoseconds); it determines when the value was recorded and is the primary index

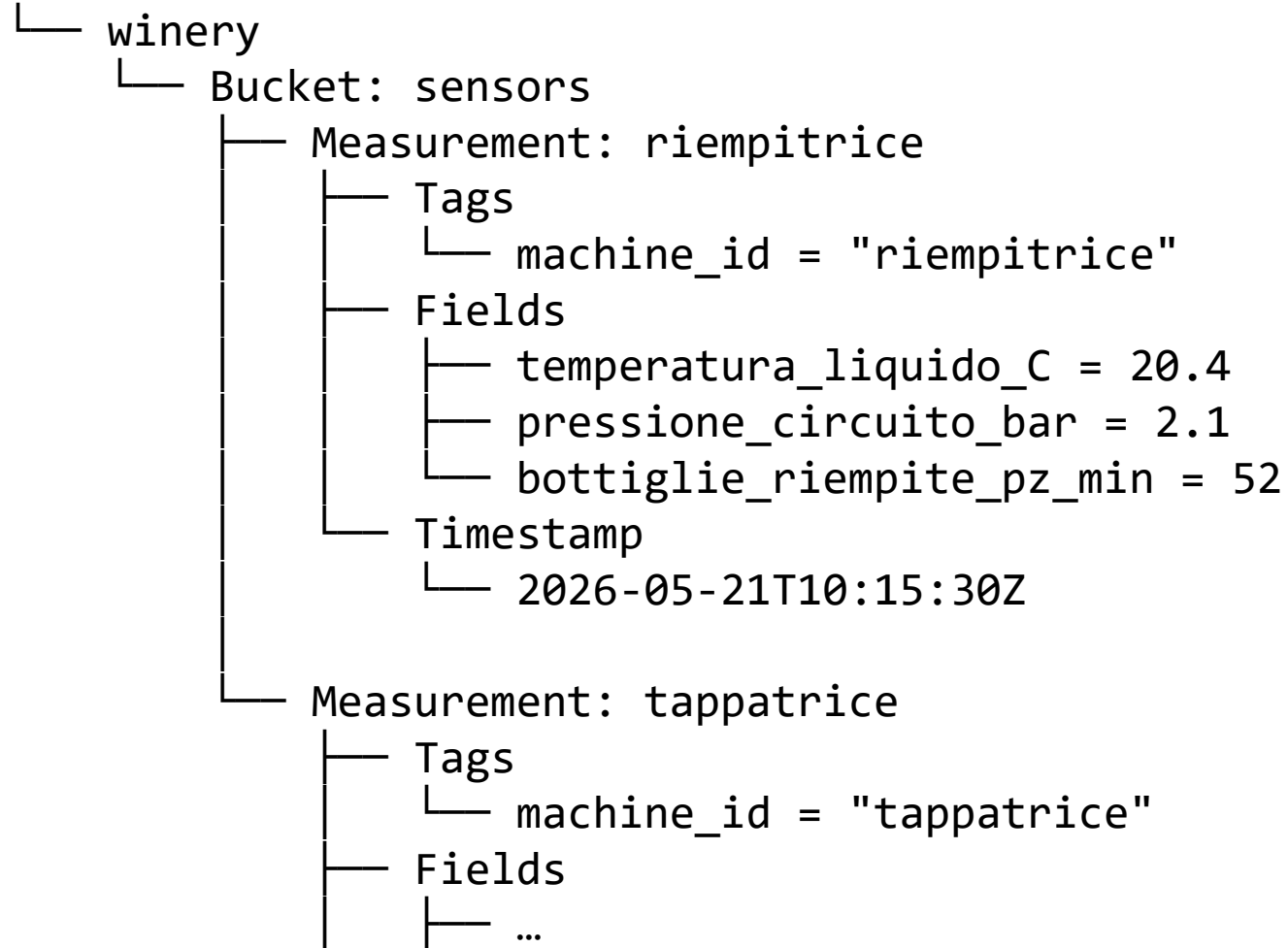
Exercise

- I have 3 sensors at home, each monitoring temperature and pressure. They are in the kitchen, living room, and restroom, respectively. How can I structure an InfluxDB database for these data?

Element	Example	Purpose
Bucket	home_sensors	Top-level storage container
Measurement	environment	Type of data
Tags	room, sensor_id	Metadata, for filtering
Fields	temperature, pressure	Actual data
Timestamp	2025-05-28T12:00:00Z	When the data was recorded

Influxdb logical model for wine factory

Organization



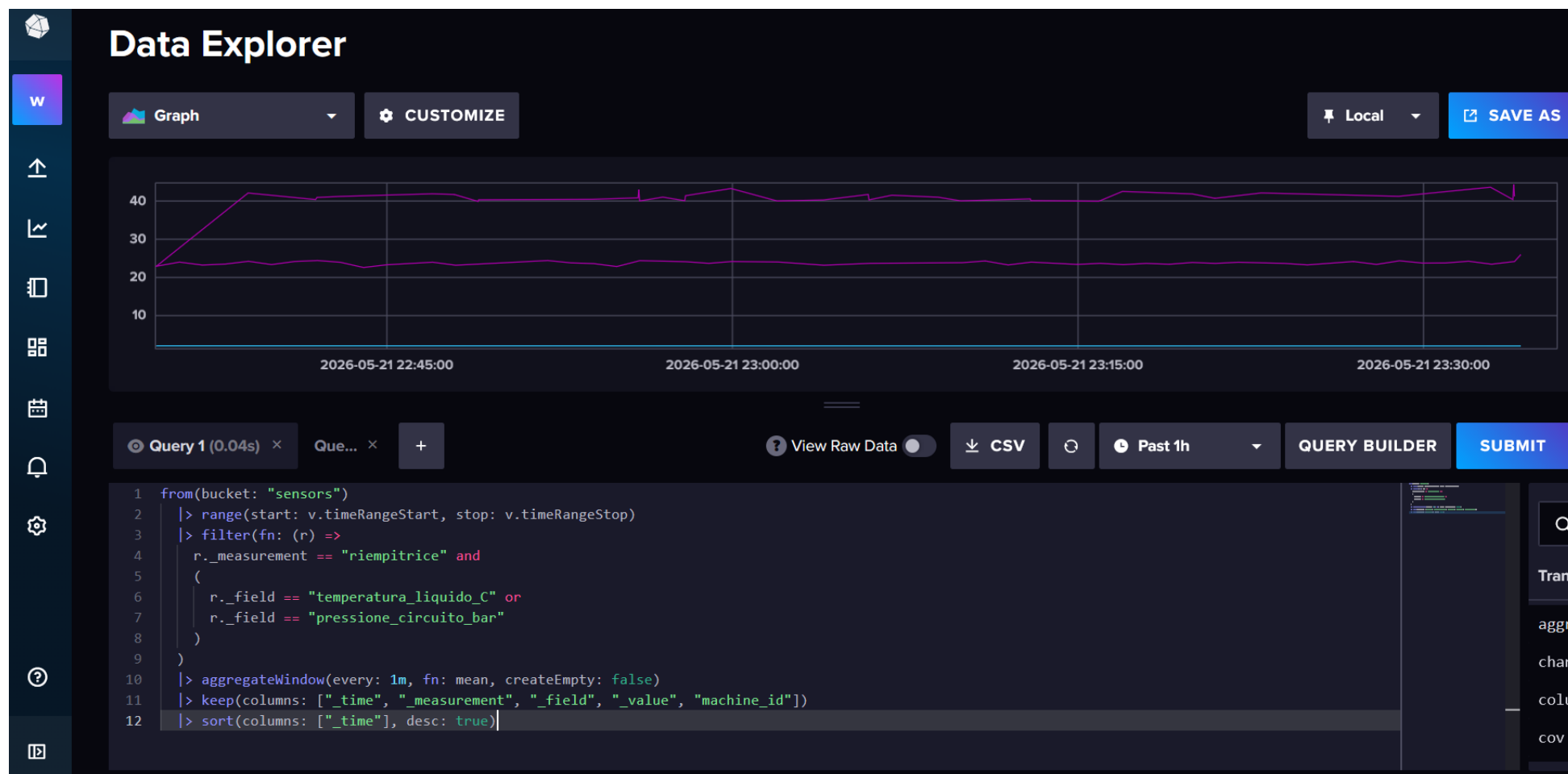
Exercise

Model an InfluxDB database considering a wine factory with:

- 2 filling machines,
- 2 capping machines, and
- 2 labeling machines

```
Organization
├── winery
│   ├── Bucket: sensors
│   │   ├── Measurement: filling_machine
│   │   │   ├── Tags
│   │   │   │   ├── machine_id = "filling_01"
│   │   │   │   ├── machine_type = "filling"
│   │   │   │   └── line = "wine_bottling_line"
│   │   │   ├── Fields
│   │   │   │   ├── liquid_temperature_C = 20.4
│   │   │   │   ├── circuit_pressure_bar = 2.1
│   │   │   │   └── bottles_filled_per_min = 52
│   │   │   └── Timestamp
│   │   │       └── 2026-05-21T10:15:30Z
│   │   ├── Measurement: capping_machine
│   │   │   ├── Tags
│   │   │   │   └── (...)
│   │   └── Measurement: labeling_machine
│   │       ├── Tags
│   │       │   ├── machine_id = "labeling_01"
│   │       │   ├── machine_type = "labeling"
│   │       │   └── line = "wine_bottling_line"
│   │       ├── Fields
│   │       │   ├── label_roll_temperature_C = 32.1
│   │       │   ├── label_position_error_mm = 0.4
│   │       │   └── bottles_labeled_per_min = 50
│   │       └── Timestamp
│   │           └── 2026-05-21T10:15:30Z
```

Influxdb



Flux filter = selection

“SELECT * FROM sensors” --> `from(bucket: "sensors")`
`|> range(start: -1h)`

```
from(bucket: "sensors")
  |> range(start: v.timeRangeStart, stop:v.timeRangeStop)
  |> filter(fn: (r) =>
    r._measurement == "riempitrice" and
    (r._field == "temperatura_liquido_C" or
     r._field == "pressione_circuito_bar")
  )
```

`select only temp and
pressure from riempitrice`

`fn:` is a function declared within the query to select only certain data points

Flux filter = selection

- Select all temperatures greater than 35

```
from(bucket: "sensors")  
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)  
  |> filter(fn: (r) => r._measurement == "riempitrice")  
  |> filter(fn: (r) => r._field == "temperatura_liquido_C")  
  |> filter(fn: (r) => r._value > 35.0)
```

Flux keep = projection

- Select all temperatures greater than 30 and keep only some fields

```
from(bucket: "sensors")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "riempitrice")
  |> filter(fn: (r) => r._field == "temperatura_liquido_C")
  |> filter(fn: (r) => r._value > 30.0)
  |> keep(columns: ["_time", "_measurement", "_field",
                   "_value", "machine_id"])
```

Flux exercise

Select abnormal values: temperature above 30 °C from the filling machine or vibration above 5 mm/s from the capping machine. Return only time, measurement, field, value, and machine ID.

```
from(bucket: "sensors")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) =>
    (
      r._measurement == "riempitrice" and
      r._field == "temperatura_liquido_C" and
      r._value > 30.0
    )
    or
    (
      r._measurement == "tappatrice" and
      r._field == "vibrazione_motore_mm_s" and
      r._value > 5.0
    )
  )
  |> keep(columns: ["_time", "_measurement", "_field",
    "_value", "machine_id"])
```

Flux sort

Select abnormal temperature values from the filling machine where temperature is greater than 30 °C, considering only the last 5 minutes, and order the results from highest to lowest temperature.

```
from(bucket: "sensors")
  |> range(start: -5m)
  |> filter(fn: (r) =>
    r._measurement == "riempitrice" and
    r._field == "temperatura_liquido_C" and
    r._value > 30.0
  )
  |> keep(columns: ["_time", "_measurement",
    "_field", "_value", "machine_id"])
  |> sort(columns: ["_value"], desc: true)
```


Flux aggregateWindow

Return mean
temperature and
pressure values
over 1 minute
sliding windows
– sort newest to
oldest

```
from(bucket: "sensors")
  |> range(start: v.timeRangeStart,
          stop: v.timeRangeStop)
  |> filter(fn: (r) =>
    r._measurement == "riempitrice" and
    (
      r._field == "temperatura_liquido_C" or
      r._field == "pressione_circuito_bar"
    )
  )
  |> aggregateWindow(every: 1m,
                    fn: mean, createEmpty: false)
  |> keep(columns: ["_time", "_measurement", "_field",
                  "_value", "machine_id"])
  |> sort(columns: ["_time"], desc: true)
```

Flux aggregateWindow

aggregateWindow sorts data within each series

- not across series unless you first **ungroup** them:

```
from(bucket: "sensors")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) =>
    r._measurement == "riempitrice" and (
      r._field == "temperatura_liquido_C" or
      r._field == "pressione_circuito_bar" )
  )
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
  |> keep(columns: ["_time", "_measurement", "_field", "_value", "machine_id"])
  |> group(columns: []) // removes grouping by _field and others
  |> sort(columns: ["_time"], desc: true)
```

Influxdb Alerts

- Rule-based notification system monitoring data streams for specific conditions
- When met, it **triggers** an action (email, posting, logging)

Component	Description
Check	Defines the logic for the alert (e.g., a threshold, deadman check, etc.)
Notification Rule	Specifies how and when to send alerts (email, Slack, webhook, etc.)
Notification Endpoint	The destination (e.g., Slack URL, email address)
Task	Background process that evaluates the alert rule periodically

Influxdb Alerts

- Types of alerts:
 - Threshold
 - Deadman: if data stops arriving
 - Custom: built with Flux queries
- How to create an Alert that sends a Slack message when average wine temperature is larger than 35 in the last 5 seconds

Steps to program an Alert

1. **Select the metric to monitor** - Example: liquid temperature from the filling machine
2. **Choose the evaluation window** - Example: last 5 seconds
3. **Choose the aggregation function** - Example: compute the mean temperature over that window

Steps to program an Alert

4. **Define the threshold condition** - Example: alert if the mean temperature is greater than 30 °C
5. **Create the check** -The check evaluates the condition and produces a status such as ok, warn, or crit

Steps to program an Alert

6. **Create the notification endpoint** - Example: Slack webhook, email, or HTTP webhook
7. **Create the notification rule** - Example: send a notification only when the status is crit; schedule the check every 3 seconds
8. **Format the notification message** - Include the machine, metric, aggregation, value, threshold, and status

Create the Check

Winery_check

1. DEFINE QUERY

2. CONFIGURE CHECK



To create a threshold check, you must select:

- ✓ One field
- ✓ One aggregate function
- ✗ One or more thresholds

Query 1 (0.02s) View Raw Data SUBMIT

FROM

Search buckets

sensors

_monitoring

_tasks

+ Create Bucket

Filter

_measurement

1

Search _measurement tag values

riempitrice

tappatrice

Filter

_field

1

Search _field tag values

bottiglie_riempite_pz_min

pressione_circuito_bar

temperatura_liquido_C

WINDOW PERIOD

auto (1m)

AGGREGATE FUNCTION

mean

median

max

min

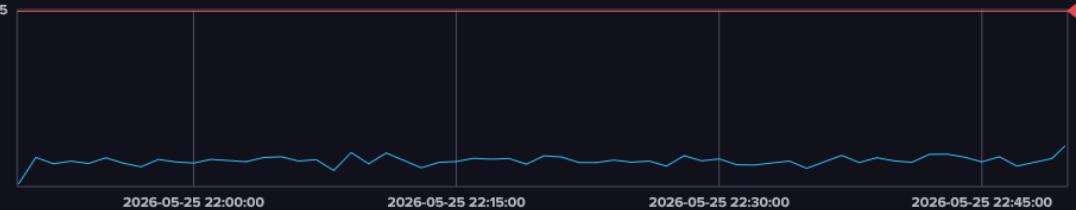
sum

derivative

Winery_check

1. DEFINE QUERY

2. CONFIGURE CHECK



Configure a Check View Raw Data HELP

PROPERTIES

Schedule Every 5s

Offset 0s

Tags

+ Tag

STATUS MESSAGE TEMPLATE

High temperature alert on riempitrice | Mean temperature: \${ r.temperatura_liquido_C } °C | Threshold: 30 °C | Status: \${ r._level }

You can use any columns from your query as well as the following:

\${ r._check_name } The name of this check

\${ r._level } Indicates the level of the check

\${ r._string(r._numericColumn) } Functions

THRESHOLDS

When value 35 is above

set status to CRIT

+ WARN

+ INFO

Create the Endpoint

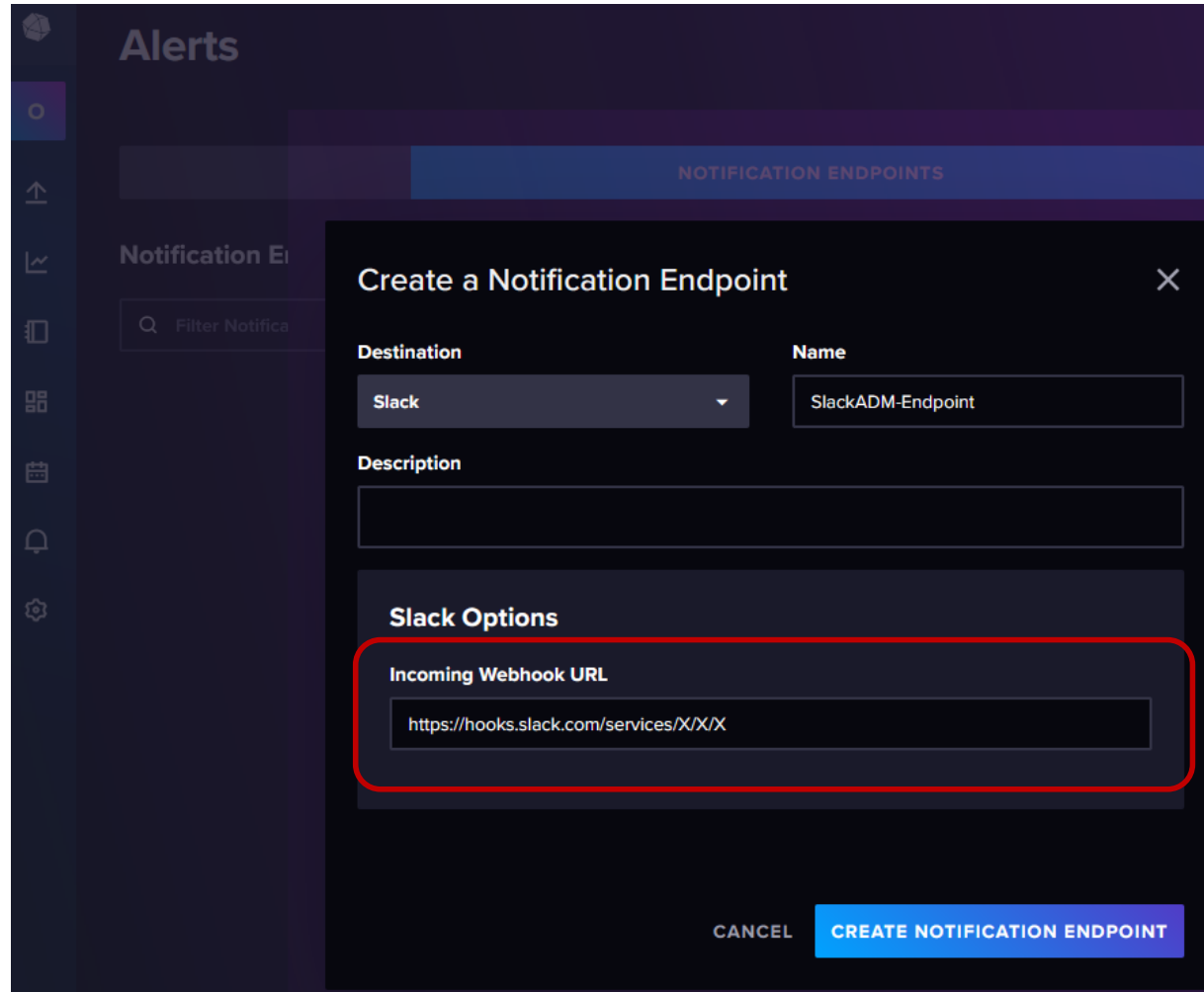
Create a new Slack Workplace

Create a new Slack app named «control-room»

- Go to <https://api.slack.com/apps>
- Create a new app “from scratch” → select "Incoming Webhooks"
- Enable Webhooks and add one to a channel
- Copy the Webhook URL

Create the Notification Endpoint

Paste the
Webhook
URL here:



The screenshot shows a dark-themed application interface. On the left is a sidebar with icons for Alerts, Notification Endpoints, and other features. The main area displays a modal titled "Create a Notification Endpoint" with a close button (X) in the top right corner. The modal contains the following fields:

- Destination:** A dropdown menu with "Slack" selected.
- Name:** A text input field containing "SlackADM-Endpoint".
- Description:** An empty text input field.
- Slack Options:** A section containing an "Incoming Webhook URL" text input field with the placeholder text "https://hooks.slack.com/services/X/X/X". This section is highlighted with a red rounded rectangle.

At the bottom of the modal are two buttons: "CANCEL" and "CREATE NOTIFICATION ENDPOINT".

Create the Notification Rule

Edit this Notification Rule

About

Name

Winery_notification_rule

Schedule Every

3s

Offset

0s

Conditions

When status

is equal to

CRIT

+ Tag Filter

Message

Notification Endpoint

Winery_slack_notification_endpoint

Message Template

\$(r_message)

Slack notification

The screenshot displays the Slack web interface. On the left is a dark purple sidebar with navigation options: Home, More, Admin, and a list of channels including #winery_alerts. The main workspace shows the #winery_alerts channel. A yellow notification bubble is highlighted, containing the text: "High temperature alert on riempitrice | Mean temperature: 36.98 °C | Threshold: 35 °C | Status: crit | Time: 2026-05-25T21:14:10.000000000Z". Above this bubble, a "Status: crit" label is visible. Below the notification is a "Welcome to #winery_alerts" message with a star icon and a dropdown menu set to "All new posts". At the bottom, there is a text input area with various formatting and action icons.

ADM

Upgrade Plan

Huddles

Directories

Starred

Drag and drop important stuff here

Channels

winery_alerts

Direct messages

Apps

Slackbot

winery

Search ADM

winery_alerts

Invite teammates

Messages

Add canvas

Status: crit

Today

High temperature alert on riempitrice | Mean temperature: 36.98 °C | Threshold: 35 °C | Status: crit | Time: 2026-05-25T21:14:10.000000000Z

Welcome to #winery_alerts

Group this channel with similar conversations in your sidebar and choose when you want to be notified about new messages.

All new posts

Message #winery_alerts

Grafana

- **Visualization Platform:** Grafana turns time-series data from InfluxDB into interactive dashboards and visual insights
- **Custom Dashboards & Panels:** Offers flexible, user-defined dashboards with charts, graphs, heatmaps, and alerts
- **Alerting & Monitoring:** Supports configurable alerts based on InfluxDB queries, with notifications via email, Slack, and more
- **User Access & Sharing:** Enables multi-user access with permissions and easy sharing of dashboards across teams

Connect Grafana to Influxdb

The screenshot shows the Grafana web interface with the 'Data sources' menu item selected in the left sidebar. The main content area is titled 'Query language' and shows 'Flux' selected in a dropdown menu. Below this, a blue informational box states: 'Support for Flux in Grafana is currently in beta. Please report any issues to: <https://github.com/grafana/grafana/issues>'. The 'HTTP' section is expanded, showing the 'URL' field with the value 'http://influxdb:8086'. Below the URL, there is a section for 'Allowed cookies' with a text input 'New tag (enter key to add)' and an 'Add' button. The 'Timeout' section shows a text input for 'Timeout in seconds'.

Home

Bookmarks

Starred

Dashboards

Explore

Drilldown New!

Alerting

Connections

Add new connection

Data sources

Administration

Query language

Flux

Support for Flux in Grafana is currently in beta
Please report any issues to:
<https://github.com/grafana/grafana/issues>

HTTP

URL
Your access method is *Server*, this means the URL needs to be accessible from the grafana backend/server.

http://influxdb:8086

Allowed cookies
Grafana proxy deletes forwarded cookies by default. Specify cookies by name that should be forwarded to the data source.

New tag (enter key to add) Add

Timeout
HTTP request timeout in seconds

Timeout in seconds

Connect Grafana to Influxdb

Auth

Basic auth	☐	With Credentials	☐
TLS Client Auth	☐	With CA Cert	☐
Skip TLS Verify	☒		
Forward OAuth Identity	☐		

Custom HTTP Headers

+ Add header

InfluxDB Details

Organization	winery	
Token	configured	Reset
Default Bucket	sensors	
Min time interval	10s	
Max series	1000	

Panels



Real-time aggregations to MongoDB

Telegraf.conf

Telegraf sends aggregated data on-the-fly to MongoDB to permanently store long-term statistics

```
# Aggregate metrics before sending to MongoDB
[[aggregators.basicstats]]
  namepass = ["riempitrice", "tappatrice"]
  period = "60s"
  drop_original = false
  stats = ["mean"]
  [aggregators.basicstats.tags]
    aggregated = "true"

[[outputs.mongodb]]
  dsn = "mongodb://admin:adminpassword@winery-mongodb:27017/?authSource=admin"
  database = "winery"
  granularity = "seconds"
  ttl = "720h"
  [outputs.mongodb.tagpass]
    aggregated = ["true"]
```

Aggregated historical data in MongoDB

The screenshot displays the MongoDB Compass interface. On the left, the 'CONNECTIONS (3)' panel shows a tree view of connections. Under the 'winery' connection, the 'riempitrice' database is selected, showing collections 'sensor_history' and 'tappatrice'. Below this, 'localhost:27017' is listed twice. The main panel on the right shows a query bar with the text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar are buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT DATA'. The query results are displayed in two rows. Each row shows a document with the following fields: 'timestamp', 'tags' (an object), 'pressione_circuito_bar', '_id' (an ObjectId), 'temperatura_liquido_C', and 'bottiglie_riempite_pz_min'.

CONNECTIONS (3)

Search connections

- ▼ Docker
 - ▶ admin
 - ▶ config
 - ▶ local
- ▼ winery
 - riempitrice
 - sensor_history
 - tappatrice
- ▶ localhost:27017
- ▶ localhost:27017

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA **UPDATE** **DELETE** **EXPORT DATA**

Document 1:

```
{
  timestamp: 2026-05-21T14:15:28.952+00:00,
  tags: Object,
  pressione_circuito_bar: 1.674,
  _id: ObjectId('6a0f1382d7a9f59400552c3c'),
  temperatura_liquido_C: 19.32,
  bottiglie_riempite_pz_min: 50
}
```

Document 2:

```
{
  timestamp: 2026-05-21T14:15:46.599+00:00,
  tags: Object,
  pressione_circuito_bar: 1.602,
  _id: ObjectId('6a0f1397d7a9f59400552c4f'),
  temperatura_liquido_C: 21.05,
  bottiglie_riempite_pz_min: 50
}
```

Add Mosquitto to the Docker container

1. Add these volumes in docker-compose.yml (under volumes:)

```
mosquitto-data  
mosquitto-log
```

2. Add Mosquitto service:

```
mosquitto:  
  image: eclipse-mosquitto:2  
  container_name: winery-mosquitto  
  networks:  
    - winery-net  
  ports:  
    - "1883:1883"  
  volumes:  
    - ./mosquitto/config:/mosquitto/config:ro  
    - mosquitto-data:/mosquitto/data  
    - mosquitto-log:/mosquitto/log  
  restart: on-failure
```

Add Mosquitto to the Docker container

3. Add file: mosquitto/config/mosquitto.conf

```
persistence true
persistence_location /mosquitto/data/

log_dest stdout
log_dest file /mosquitto/log/mosquitto.log

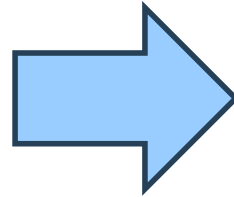
listener 1883 0.0.0.0
allow_anonymous true
```

4. Restart the container with:

```
docker compose up -d
```

Switch the producer from Kafka to Mosquitto

```
producer:
  build:
    context: ./producer
    dockerfile: Dockerfile
  container_name: winery-producer
  networks:
    - winery-net
  depends_on:
    kafka:
      condition: service_healthy
  environment:
    KAFKA_BOOTSTRAP_SERVERS: winery-kafka:29092
    TOPIC_RIEMPITRICE: sensors.riempitrice
    TOPIC_TAPPATRICE: sensors.tappatrice
    PUBLISH_INTERVAL_SECONDS: "2"
  restart: on-failure
```



```
producer:
  build:
    context: ./producer
    dockerfile: Dockerfile
  container_name: winery-producer
  networks:
    - winery-net
  depends_on:
    - mosquitto
  environment:
    MQTT_HOST: mosquitto
    MQTT_PORT: "1883"
    TOPIC_RIEMPITRICE: sensors.riempitrice
    TOPIC_TAPPATRICE: sensors.tappatrice
    PUBLISH_INTERVAL_SECONDS: "2"
  restart: on-failure
```

Switch the producer from Kafka to Mosquitto

- Add paho-mqtt to requirements.txt
- In docker-compose.yml add:

 telegraf:
 (...)
 depends_on:
 - mosquitto
- Put producer-mqtt.py in /producer

Switch the producer from Kafka to Mosquitto

Add these lines in telegraf.conf:

```
[[inputs.mqtt_consumer]]
  servers = ["tcp://mosquitto:1883"]
  topics = ["sensors.riempitrice"]
  client_id = "telegraf-riempitrice"
  qos = 1
  connection_timeout = "30s"
  name_override = "riempitrice"
  data_format = "json_v2"
  topic_tag = ""

[[inputs.mqtt_consumer.json_v2]]
  [[inputs.mqtt_consumer.json_v2.tag]]
    path = "machine_id"
```

(cont.)

```
[[inputs.mqtt_consumer.json_v2.field]]
  path = "temperatura_liquido_C"
  type = "float"

[[inputs.mqtt_consumer.json_v2.field]]
  path = "pressione_circuito_bar"
  type = "float"

[[inputs.mqtt_consumer.json_v2.field]]
  path = "bottiglie_riempite_pz_min"
  type = "float"
```

Switch the producer from Kafka to Mosquitto

Add these lines in telegraf.conf:

```
[[inputs.mqtt_consumer]]
  servers = ["tcp://mosquitto:1883"]
  topics = ["sensors.tappatrice"]
  client_id = "telegraf-tappatrice"
  qos = 0
  connection_timeout = "30s"
  name_override = "tappatrice"
  data_format = "json_v2"
  topic_tag = ""

[[inputs.mqtt_consumer.json_v2]]
  [[inputs.mqtt_consumer.json_v2.tag]]
    path = "machine_id"
```

(cont.)

```
[[inputs.mqtt_consumer.json_v2.field]]
  path = "forza_tappatura_N"
  type = "float"

[[inputs.mqtt_consumer.json_v2.field]]
  path = "vibrazione_motore_mm_s"
  type = "float"

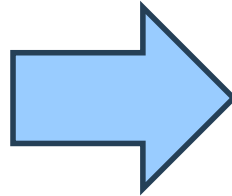
[[inputs.mqtt_consumer.json_v2.field]]
  path = "bottiglie_tappate_pz_min"
  type = "float"
```


Build and restart the container

In producer/Dockerfile:

```
COPY producer.py .
```

```
CMD ["python", "-u",  
     "producer.py"]
```



```
COPY producer-mqtt.py .
```

```
CMD ["python", "-u",  
     "producer-mqtt.py"]
```

```
docker compose down  
docker compose up -d --build
```

The rest of the pipeline continues to work unchanged